

```

import time
import board
import adafruit_dht
import requests
import socket
import threading
import cv2
import serial
import pynmea2
import os

from flask import Flask, Response, jsonify
from gpiozero import DigitalInputDevice, DigitalOutputDevice, Button

# ----- GPIO CONFIG -----
IR_PIN, MQ2_PIN, FAN_PIN, BIKE_PIN, SOS_PIN = 17, 27, 22, 23, 24

ir_sensor = DigitalInputDevice(IR_PIN)
mq2_sensor = DigitalInputDevice(MQ2_PIN)
fan = DigitalOutputDevice(FAN_PIN)
bike_ignition = DigitalOutputDevice(BIKE_PIN)
sos_button = Button(SOS_PIN, pull_up=True)

# ----- DHT SENSOR -----
dht_device = adafruit_dht.DHT11(board.D18)

# ----- TELEGRAM CONFIG -----
BOT_TOKEN = "8744050249:AAE-r5rzpLN9hXr7PBPaIw2XPOD-63LPrl"
CHAT_ID = "5507854201"

# ----- GLOBAL STATE -----
gps_lat = 0
gps_lon = 0
gps_fix = False

system_data = {
    "temperature": 0,
    "humidity": 0,
    "helmet": False,
    "alcohol": False,
    "bike_active": False,
    "fan_active": False,
    "status_msg": "Initializing System...",
    "gps_lat": 0,
    "gps_lon": 0,
    "gps_fix": False
}

```

```

app = Flask(__name__)

# ----- CAMERA SETUP -----
camera = cv2.VideoCapture(0)
camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# ----- GPS THREAD -----
def gps_loop():
    global gps_lat, gps_lon, gps_fix
    try:
        ser = serial.Serial('/dev/serial0', 9600, timeout=1)
        while True:
            line = ser.readline().decode('ascii', errors='replace')
            if "$GPGGA" in line:
                try:
                    msg = pynmea2.parse(line)
                    if msg.latitude != 0:
                        gps_lat = msg.latitude
                        gps_lon = msg.longitude
                        gps_fix = True
                except:
                    pass
    except Exception as e:
        print(f"GPS Serial Error: {e}")

# ----- SOS LOGIC -----
def handle_sos():
    print("🚨 SOS TRIGGERED")
    location_url = "Location Not Available"
    if gps_fix:
        location_url = f"https://www.google.com/maps?q={gps_lat},{gps_lon}"

    success, frame = camera.read()
    if success:
        photo_path = "sos_alert.jpg"
        cv2.imwrite(photo_path, frame)

    try:
        url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendPhoto"
        caption = f"🚨 SOS EMERGENCY ALERT!\n\n📍 Location: {location_url}\n🕒 Time: {time.strftime('%H:%M:%S')}"

        with open(photo_path, 'rb') as f:
            requests.post(url, data={"chat_id": CHAT_ID, "caption": caption}, files={"photo": f},
            timeout=10)
        print("✅ SOS Sent to Telegram")
    except Exception as e:

```

```

        print(f"Telegram Error: {e}")

sos_button.when_pressed = handle_sos

# ----- CAMERA STREAM GENERATOR -----
def gen_frames():
    while True:
        success, frame = camera.read()
        if not success:
            break
        # HUD Overlay on video
        cv2.putText(frame, time.strftime("%Y-%m-%d %H:%M:%S"), (15, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

        ret, buffer = cv2.imencode('.jpg', frame)
        frame = buffer.tobytes()
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

# ----- HARDWARE MONITORING LOOP -----
def hardware_loop():
    global system_data
    while True:
        try:
            h = dht_device.humidity
            t = dht_device.temperature
            helmet_on = not ir_sensor.value
            alcohol_detected = not mq2_sensor.value

            # Fan Control (Humidity based)
            fan_state = h > 70 if h else False
            fan.on() if fan_state else fan.off()

            # Ignition Safety Logic
            bike_on = False
            if helmet_on:
                if alcohol_detected:
                    bike_ignition.off()
                    msg = "IGNITION BLOCKED: ALCOHOL"
                else:
                    bike_ignition.on()
                    bike_on = True
                    msg = "ENGINE RUNNING"
            else:
                bike_ignition.off()
                msg = "WAITING: WEAR HELMET"

            system_data.update({

```

```

        "temperature": t if t else 0,
        "humidity": h if h else 0,
        "helmet": helmet_on,
        "alcohol": alcohol_detected,
        "bike_active": bike_on,
        "fan_active": fan_state,
        "status_msg": msg,
        "gps_lat": gps_lat,
        "gps_lon": gps_lon,
        "gps_fix": gps_fix
    })
except Exception as e:
    pass
time.sleep(1)

```

----- WEB INTERFACE (CYBERPUNK UI) -----

```

@app.route('/')
def index():
    return """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Guardian OS | Smart Helmet</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css"
rel="stylesheet">
    <style>
        @import
url('https://fonts.googleapis.com/css2?family=Orbitron:wght@400;700&family=Rajdhani:wght
@500;700&display=swap');
        body { background: radial-gradient(circle at center, #0f172a 0%, #020617 100%); color:
#e2e8f0; font-family: 'Rajdhani', sans-serif; }
        .orbitron { font-family: 'Orbitron', sans-serif; }
        .glass { background: rgba(30, 41, 59, 0.4); backdrop-filter: blur(12px); border: 1px solid
rgba(255, 255, 255, 0.1); }
        .status-glow { width: 12px; height: 12px; border-radius: 50%; display: inline-block;
margin-right: 8px; }
        .pulse { animation: pulse-animation 2s infinite; }
        @keyframes pulse-animation { 0% { box-shadow: 0 0 0 0px rgba(56, 189, 248, 0.4); }
100% { box-shadow: 0 0 0 15px rgba(56, 189, 248, 0); } }
        .wheel { transform-origin: center; transform-box: fill-box; }
        .bike-active .wheel { animation: spin 0.8s linear infinite; }
        @keyframes spin { from { transform: rotate(0deg); } to { transform: rotate(360deg); } }
    </style>
</head>
<body class="p-4 md:p-8">

```

```

<div class="max-w-6xl mx-auto">
  <header class="flex flex-col md:flex-row justify-between items-center mb-10 gap-4">
    <div>
      <h1 class="text-4xl font-bold orbitron tracking-tighter text-sky-400
uppercase">Guardian OS</h1>
      <p class="text-slate-400 tracking-widest text-sm">RIDER TELEMETRY HUB</p>
    </div>
    <div class="glass px-6 py-2 rounded-full border border-sky-500/30">
      <span id="main-dot" class="status-glow bg-sky-400 pulse"></span>
      <span id="main-status" class="orbitron text-sm uppercase tracking-widest
text-sky-400">System Ready</span>
    </div>
  </header>

  <div class="grid grid-cols-1 lg:grid-cols-3 gap-8">
    <div class="space-y-6">
      <div class="glass p-6 rounded-2xl border-l-4 border-sky-500">
        <p class="text-slate-400 text-xs uppercase">Internal Temp</p>
        <h2 id="temp" class="text-4xl font-bold orbitron mt-1">0°C</h2>
      </div>
      <div class="glass p-6 rounded-2xl border-l-4 border-blue-500">
        <p class="text-slate-400 text-xs uppercase">Humidity</p>
        <h2 id="hum" class="text-4xl font-bold orbitron mt-1">0%</h2>
      </div>
      <div class="glass p-6 rounded-2xl">
        <h3 class="orbitron text-sm text-sky-400 mb-4 uppercase">Navigation</h3>
        <p id="gps-coords" class="text-lg font-bold">Searching GPS...</p>
        <a id="map-link" href="#" target="_blank" class="block mt-4 w-full text-center
bg-sky-600 py-2 rounded-lg font-bold opacity-50 pointer-events-none transition-all">OPEN
MAP</a>
      </div>
    </div>

    <div class="lg:col-span-2 space-y-6">
      <div class="glass p-2 rounded-3xl overflow-hidden relative">
        
      </div>
      <div class="grid grid-cols-2 md:grid-cols-4 gap-4 text-center">
        <div id="card-helmet" class="glass p-4 rounded-xl border-b-2
border-transparent">
          <p class="text-[10px] text-slate-400">HELMET</p>
          <p id="txt-helmet" class="text-sm font-bold">MISSING</p>
        </div>
        <div id="card-alcohol" class="glass p-4 rounded-xl border-b-2
border-transparent">
          <p class="text-[10px] text-slate-400">SOBER</p>
          <p id="txt-alcohol" class="text-sm font-bold">CLEAN</p>
        </div>
      </div>
    </div>
  </div>

```

```

    <div id="card-bike" class="glass p-4 rounded-xl border-b-2 border-transparent">
      <p class="text-[10px] text-slate-400">ENGINE</p>
      <p id="txt-bike" class="text-sm font-bold">LOCKED</p>
    </div>
    <div id="card-fan" class="glass p-4 rounded-xl border-b-2 border-transparent">
      <p class="text-[10px] text-slate-400">FAN</p>
      <p id="txt-fan" class="text-sm font-bold">OFF</p>
    </div>
  </div>
</div>
</div>
</div>
</div>

<script>
  async function update() {
    try {
      const res = await fetch('/sensor-data');
      const d = await res.json();
      document.getElementById('temp').innerText = d.temperature + "°C";
      document.getElementById('hum').innerText = d.humidity + "%";
      document.getElementById('main-status').innerText = d.status_msg;

      if(d.gps_fix) {
        document.getElementById('gps-coords').innerText = d.gps_lat.toFixed(5) + ", " +
d.gps_lon.toFixed(5);
        const map = document.getElementById('map-link');
        map.href = `https://www.google.com/maps?q=${d.gps_lat},${d.gps_lon}`;
        map.classList.remove('opacity-50', 'pointer-events-none');
      }

      updateCard('card-helmet', 'txt-helmet', d.helmet, 'SECURED', 'MISSING');
      updateCard('card-alcohol', 'txt-alcohol', !d.alcohol, 'CLEAN', 'DANGER', true);
      updateCard('card-bike', 'txt-bike', d.bike_active, 'RUNNING', 'LOCKED');
      updateCard('card-fan', 'txt-fan', d.fan_active, 'ACTIVE', 'OFF');
    } catch(e) {}
  }
  function updateCard(cid, tid, active, t1, t2, inv=false) {
    const c = document.getElementById(cid);
    const t = document.getElementById(tid);
    c.style.borderColor = active ? "#4ade80" : (inv ? "#f43f5e" : "transparent");
    t.innerText = active ? t1 : t2;
    t.className = "text-sm font-bold " + (active ? "text-green-400" : (inv ? "text-red-500" :
"text-slate-500"));
  }
  setInterval(update, 1000);
</script>
</body>
</html>

```

"""

```
@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')
```

```
@app.route('/sensor-data')
def sensor():
    return jsonify(system_data)
```

```
if __name__ == "__main__":
    threading.Thread(target=hardware_loop, daemon=True).start()
    threading.Thread(target=gps_loop, daemon=True).start()
    print("🚀 SYSTEM STARTED AT PORT 8000")
    app.run(host='0.0.0.0', port=8000, debug=False)
```